

代码反推基于0429代码，使用轻度搜索

执行摘要

1. 符号与问题定义

令 $X \in \mathbb{R}^{N \times F}$ 为节点属性矩阵（ N 个节点，特征维度 F ），原始超图结构用二元组列表或关联矩阵 $H \in 0,1^{N \times M}$ 表示（ M 条超边）。算法的目标是学习节点嵌入 $Z = f_\theta(X, H) \in \mathbb{R}^{N \times d}$ 使之适用于聚类，其中 f_θ 为 EnhancedHGNN 编码器（第2节）。给定真实类别数 K （在实验中取自标签，不参与训练），维护可学习聚类原型矩阵 $C \in \mathbb{R}^{K \times d}$ （对应 ClusterPrototypes 模块）。以下使用 $\bar{z}_i = z_i / \|z_i\|_2$ 表示归一化后节点嵌入，第 i 行记为 $\bar{z}_i^\top$ 。另外，超参数有学习率、温度系数 τ, τ_c ，损失权重 $\alpha, \beta, \gamma, \delta$ 等（见附表）。所有可学习参数包括编码器参数 θ 、投影头参数 ϕ 、聚类原型 C 及损失模块权重等。

2. 节点表示学习（编码器）

编码器 EnhancedHGNN 首先对节点特征做线性映射：

$$\mathbf{P}^{(0)} = XW_x + b_x,$$

其中 $W_x \in \mathbb{R}^{F \times d}$ 、 $b_x \in \mathbb{R}^d$ （代码中对应 `self.lin_input`）。超边表示初始化为超边中节点表示的均值：

$$\mathbf{Q}_j^{(0)} = \frac{1}{|e_j|} \sum_{v_i \in e_j} P_i^{(0)},$$

对应代码 EnhancedHGNN 中对每个超边求节点特征均值。随后进行多层超图注意力传播：每层对节点和超边各做线性变换，再进行超边到节点和节点到超边的注意力聚合（参见 EnhancedHGNNConv）。比如，一层节点更新可写为

$$\tilde{p}_i^{(\ell)} = p_i^{(\ell)} W_P^{(\ell)}, m_i^{E \rightarrow V} = \sum_{e_j \in \mathcal{E}(i)} \alpha_{ij}^{E \rightarrow V} \tilde{q}_j^{(\ell)}, p_i^{(\ell+1)} = \text{LayerNorm}(\tilde{p}_i^{(\ell)} + m_i^{E \rightarrow V}).$$

其中注意力系数 $\alpha_{ij}^{E \rightarrow V}$ 由 $\text{LeakyReLU}(a^\top [p_i \| q_j])$ 归一化得到（代码中 `self_attn_e2v`）。最终输出 $Z = f_\theta(X, H) = P^{(L)} \in \mathbb{R}^{N \times d}$ 。该编码器是可微的，各层梯度通过残差和 LayerNorm 传播。若需要更详细公式，请参考 CAHC 相关注意力机制。

3. 数据增强与负超边

每次训练迭代使用 Augmentor 对超图做两次扰动，产生两组增强视图 $(X^{(1)}, H^{(1)})$ 和 $(X^{(2)}, H^{(2)})$ 。增强步骤包括：

- **特征掩码**：生成掩码矩阵 $M_f \in 0,1^{N \times F}$ ，每个元素以概率 p_f （代码 `feat_mask_rate`）置零。得到 $X' = X \odot M_f$ 。
- **超边扰动**：对超边关联 H 中的每条边随机删除一些节点，概率 p_e （代码 `edge_drop_rate`），得到 H' （代码 `DropIncidence`）。

此外，构造负超边集 $\mathcal{E}^-$ ：对每条超边 e_j （大小 m_j ），计算替换节点个数

$$r_j = \max(\lfloor \log_2 m_j \rfloor, \mathbf{1}_{m_j > 1}).$$

随机选取 r_j 个节点从 e_j 中删除，并随机添加同样数量其他节点，得到负超边 e_j^- （代码 `advanced_scorer` 处调用）。注意负超边仅用于结构损失计算。

4. 投影头与聚类原型

对两个增强视图 $Z^{(1)} = f_\theta(X^{(1)}, H^{(1)})$ 、 $Z^{(2)} = f_\theta(X^{(2)}, H^{(2)})$ ，分别送入投影头 g_ϕ ，得到低维投影 $P^{(1)}, P^{(2)} \in \mathbb{R}^{N \times d'}$ ；对应代码 `g_1(Z1)`、`g_2(Z2)`。同时，将原始视图嵌入 $Z = f_\theta(X, H)$ 输入聚类模块 ClusterPrototypes。该模块维护归一化原型矩阵 $C \in \mathbb{R}^{K \times d}$ ，计算节点-簇相似度：

$$r_{ik} = \frac{\bar{z}_i^\top \bar{c}_k}{\tau_c}, R = \frac{\bar{Z}, \bar{C}^\top}{\tau_c} \in \mathbb{R}^{N \times K}.$$

其中 τ_c 是聚类温度（对应代码 `self.temperature`）。软分配矩阵为

$$S_{ik} = \frac{\exp(r_{ik})}{\sum_{q=1}^K \exp(r_{iq})}, \quad S = \text{softmax}(R).$$

代码中该操作在 `ClusterPrototypes` 的 `forward` 实现。预测标签 $\hat{y}_i = \arg \max_k r_{ik}$ 用于评估但不用于训练。聚类原型可通过预训练阶段后 K-means 初始化 (`use_kmeans_init`)，否则随机初始化，均可通过梯度优化调整。

5. 损失函数

算法总损失由多项子损失加权合成：

(1)

$$\mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \gamma \mathcal{L}_{\text{cluster}} + \delta \mathcal{L}_{\text{intra}} + \lambda_{\text{LC}} (\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}}).$$

其中权重对应 `alpha, beta, gamma, delta, lnahc_weight, ortho_weight`。接下来逐项推导。

5.1 结构判别损失 $\mathcal{L}_{\text{struct}}$

构造评分器对每条超边区分真实或伪造（代码 `AdvancedScorer`）。对一条超边 e ，令节点嵌入的均值与最大池拼接

$$\mu_e = \frac{1}{|e|} \sum_{i \in e} z_i, \quad \nu_e = \max_{i \in e} (z_i), \quad h_e = [\mu_e \| \nu_e].$$

评分器输出标量分数 $s_e = \text{MLP}(h_e)$ （对应代码 `self.score`）。记真实超边集合 $\mathcal{E}^+$ ，负超边集合 $\mathcal{E}^-$ 。结构损失用对数似然：

(2)

$$\mathcal{L}_{\text{struct}} = -\frac{1}{2|\mathcal{E}^+|} \sum_{e \in \mathcal{E}^+} \log \sigma(s_e) - \frac{1}{2|\mathcal{E}^-|} \sum_{e \in \mathcal{E}^-} \log(1 - \sigma(s_e)).$$

其中 σ 是 sigmoid（代码中 `BCEWithLogits` 实现）。该损失鼓励真实超边得分增大、负超边得分减小，梯度通过 MLP 和池化传回节点嵌入 Z 。常数 $1/(2|\mathcal{E}^\pm|)$ 保证两类样本平衡。

5.2 视图对比 InfoNCE 损失 $\mathcal{L}_{\text{InfoNCE}}$

对两个增强视图投影后输出 $P^{(1)} = [p_i^{(1)}]_{i=1}^N$ 和 $P^{(2)} = [p_i^{(2)}]$ ，先归一化： $\bar{p}_i^{(v)} = p_i^{(v)} / |p_i^{(v)}|_2$ 。令相似度矩阵

(3)

$$M_{ij} = \frac{(\bar{p}_i^{(1)})^\top \bar{p}_j^{(2)}}{\tau},$$

其中温度 τ （代码 `temperature`）。然后用双向交叉熵损失对角线最大化：

(4)

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{2N} \sum_{i=1}^N \log \frac{\exp(M_{ii})}{\sum_{j=1}^N \exp(M_{ij})} - \frac{1}{2N} \sum_{i=1}^N \log \frac{\exp(M_{ii})}{\sum_{j=1}^N \exp(M_{ji})}.$$

对应代码中 `loss_fn = (CE(M,y)+CE(M^T,y))/2` (`y_i=i`)。该损失鼓励同一节点在两视图中投影相似、不同节点相异。可微性：梯度通过 $P^{(1)}, P^{(2)}$ 再传给 $Z^{(1)}, Z^{(2)}$ 和编码器。

5.3 聚类伪标签损失 $\mathcal{L}_{\text{cluster}}$

在微调阶段若 $\gamma > 0$ （`config["gamma"]`），使用当前聚类 logits R 产生伪标签。令每个节点 i 的伪标签为最大相似簇：

(5)

$$\tilde{y}_i = \arg \max_k r_{ik}.$$

然后交叉熵惩罚使预测更尖锐：

(6)

$$\mathcal{L}_{\text{cluster}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(r_{i\tilde{y}_i})}{\sum_{k=1}^K \exp(r_{ik})}.$$

该项在预训练阶段被强制为零（代码中强置 $\gamma = 0$ ）；在启用时，梯度传回聚类原型 C 和嵌入 Z ，强化已有分配。但若伪标签有误，此损失可能放大错误，因此通常配合其它正则项使用。

5.4 超边内属性指导对比损失 $\mathcal{L}_{\text{intra}}$

这是代码中针对同超边节点分布设计的自对比损失。首先构造同超边掩码矩阵 $M^H \in 0, 1^{N \times N}$ ：若节点 i, j 至少共享一条超边，则 $M_{ij}^H = 1$ ，否则 0。对应代码可写作

$$M^H = \mathbf{1}[HH^\top > 0], \quad M_{ii}^H = 1.$$

节点嵌入归一化 $\bar{z}_i = z_i / |z_i|_2$ ，定义相似度

(7)

$$a_{ij} = \frac{\bar{z}_i^\top \bar{z}_j}{\tau},$$

温度同上。若启用属性指导（`use_attr_guidance=True`），还计算原始属性归一化 $\bar{x}_i = x_i / |x_i|_2$ 的相似度 $b_{ij} = \bar{x}_i^\top \bar{x}_j$ ，并设排斥权重

(8)

$$w_{ij} = \frac{1 - b_{ij}}{2}, \quad \omega_{ij} = \{1, \quad i = j, \quad w_{ij}, \quad i \neq j \text{ (共享超边)}, 0, \quad \text{否则}.$$

即属性越不相似， w_{ij} 越大。最终损失为

(9)

$$\mathcal{L}_{\text{intra}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(a_{ii})}{\sum_{j=1}^N M_{ij}^H \omega_{ij} \exp(a_{ij}) + \varepsilon}.$$

其中 $\varepsilon = 10^{-8}$ 是数值稳定项（代码中 `eps`）。对角项 a_{ii} 保留自身，分母对节点 i 的所有同超边邻居 j 进行权重加和。若关闭属性指导，则 $\omega_{ij} = 1$ 。该损失鼓励在同一超边内部，相似节点聚拢，不相似节点远离，缓解 HGNN 的过平滑。梯度通过归一化和 softmax 路径返回到 Z 和（若启用） X 。

5.5 隐空间动态图割损失 $\mathcal{L}_{\text{cut}}$

此项对应 LNAHC/LANHC 模块，实际实现为在当前隐空间嵌入 Z 上构建动态亲和图并做 normalized cut 优化。具体步骤（代码 `compute_latent_lnahc_cut_loss`）如下：

1. **动态图构造**：首先根据 `detach_affinity` 决定是否阻断梯度。若 `detach_affinity=True`，则

$$Z_g = \text{stopgrad}(Z),$$

否则 $Z_g = Z$ 。对所有节点归一化 $\tilde{z}_i^g = z_i^g / |z_i^g|_2$ ，计算余弦相似度 $c_{ij} = (\tilde{z}_i^g)^\top \tilde{z}_j^g$ 。阈值取 $\eta = 0.5$ （代码常数），构造邻接矩阵

(10)

$$A_{ij} = \{c_{ij}, \quad i \neq j \text{ 且 } c_{ij} > \eta, 0, \quad \text{否则}, \quad A_{ii} = 0.$$

只有相似度超过 0.5 的节点对才连边（边权即相似度）。此 $A = A(Z)$ 是可微的（若未 `detach`）。

2. **度矩阵**：计算节点度 $d_i = \sum_j A_{ij}$ ，引入稳定项

(11)

$$\tilde{d}_i = d_i + 10^{-4} \cdot \frac{1}{N} \sum_p d_p, \quad D = \text{diag}(\tilde{d}_1, \dots, \tilde{d}_N).$$

该 10^{-4} 项（代码中写死）防止矩阵奇异。

3. **簇体积与关联**：令前节得到的软分配矩阵 $S \in \mathbb{R}^{N \times K}$ ，定义

(12)

$$V = S^\top D S, \quad B = S^\top A S.$$

如果 S 是 one-hot, 这两项分别近似簇的体积 ($\text{vol}(C_k) = \sum_{i \in C_k} d_i$) 和簇内保留的亲总和。通俗地, V_{kk} 衡量簇 k 吸收的总连接强度, B_{kk} 是簇内部边权。

4. **soft cut 矩阵**: 图 Laplacian $L_A = D - A$, 则

(13)

$$C_{\text{cut}} = S^\top L_A S = S^\top (D - A) S = V - B.$$

C_{cut} 实际上捕捉了簇内体积减簇内保留连接的差异, 即被切断的权重。

5. **归一化割损失**: 最终计算 normalized cut 迹式 (对应代码 `torch.linalg.pinv`):

(14)

$$\mathcal{L}_{\text{cut}} = \text{Tr}[(V + 10^{-4}I)^+, C_{\text{cut}}] = \text{Tr}[(S^\top D S + 10^{-4}I)^+, S^\top (D - A) S],$$

其中 $(\cdot)^+$ 表示 Moore–Penrose 伪逆 (代码加入 $10^{-4}I$ 防止奇异)。对于硬分配 S , 该项近似 $\sum_k \text{cut}(C_k, \bar{C}_k) / \text{vol}(C_k)$, 即归一化割指标。该损失鼓励: **隐空间亲和强的节点应被同簇分配, 减少被切割**。等价地可证明 (见附注):

(15)

$$\text{Tr}(S^\top (D - A) S) = \frac{1}{2} \sum_{i,j} A_{ij} |s_i - s_j|_2^2.$$

(当 S 为概率分配时近似成立), 前提是 A 对称且分配向量 s_i 包含各簇概率。此式表明: 若 A_{ij} 很大, 节点 i, j 的分配 s_i, s_j 应尽量相似, 否则 $|s_i - s_j|^2$ 代价高。梯度流向: 若未阻断, $\mathcal{L}_{\text{cut}}$ 通过 A 和 S 同时影响 Z ; 若 `detach_affinity=True`, 则 A 视为常数, 梯度仅通过 S (即簇原型和 Z) 回传。**梯度流向示意图见下文第7节**。

5.6 平衡项 $\mathcal{L}_{\text{balance}}$

为防止隐空间划分退化, 代码提供两种平衡约束 (`balance_loss_type`)。当设为 "ortho" 时, 使用正交均衡项:

(16)

$$\mathcal{L}_{\text{ortho}} = \frac{1}{N} S^\top S - \frac{1}{K} I_K^2.$$

其中 $S^\top S \in \mathbb{R}^{K \times K}$, 目标矩阵为 $\frac{1}{K} I_K$ (代码中 `self.ortho_target=1/K*I`)。此项同时保证不同簇尽可能正交 (非对角元趋0) 且每簇概率总和接近 N/K , 从而鼓励簇大小均衡。

若设为 "empty", 使用空簇惩罚: 定义每簇平均分配 $m_k = \frac{1}{N} \sum_i S_{ik}$, 给定阈值 ρ (默认为 0.02), 损失为

(17)

$$\mathcal{L}_{\text{empty}} = \frac{1}{K} \sum_{k=1}^K [\max(0, \rho - m_k)]^2.$$

该项只对过空的簇 ($m_k < \rho$) 施加惩罚, 弱于严格均衡。最终使用 $\mathcal{L}_{\text{balance}}$ 表示选取的方案。

结合切割和均衡项, 代码实际使用 $\lambda_{\text{LC}}(\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}}\mathcal{L}_{\text{balance}})$, 其中 $\lambda_{\text{LC}} = \text{lnahc_weight}$, $\lambda_{\text{bal}} = \text{ortho_weight}$ 。如果 `lnahc_weight=0` 则跳过整块优化。

6. 训练阶段: 预训练与微调

训练分两阶段 (代码中由 `pretrain_epochs` 和 `finetune_epochs` 控制):

- **预训练阶段**: 在前期训练中强制设 $\gamma = 0$ 和 $\lambda_{\text{LC}} = 0$ (代码直接将 `current_gamma=0`, `current_lnahc_weight=0`)。故预训练损失为 (18)

$$\mathcal{L}_{\text{pre}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \delta \mathcal{L}_{\text{intra}}.$$

此时只优化结构判别、视图对比和同超边内对比，主要学习表示和保持增强一致性、保留异质性。聚类原型虽被创建和更新，但未参与损失。预训练结束后，若 `use_kmeans_init=True`，代码使用预训练嵌入运行 K-Means：令 $Z_{\text{pre}} = f_{\theta}(X, H)$ ，则

$$C^{(0)} = \text{KMeans}(Z_{\text{pre}}, K)$$

得到 K 个簇中心，用以初始化原型矩阵 C 。这一步**不当作最终输出**，仅为微调提供稳健起点。

- **微调阶段**：进入第二阶段，从配置读取 γ 和 λ_{LC} ，构造总损失 (19)

$$\mathcal{L}_{\text{fine}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \gamma \mathcal{L}_{\text{cluster}} + \delta \mathcal{L}_{\text{intra}} + \lambda_{\text{LC}} (\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}}).$$

此时聚类伪标签和隐空间切割损失加入优化。梯度同时更新编码器参数 θ 、聚类原型 C 、投影头 ϕ 以及损失模块（如评分器和平衡权重等）。微调结束后，使用最后一轮模型输出的 $\hat{y}_i$ 作为最终聚类结果（代码中仅使用 `cluster_module_final`，而非 `oracle` 调整）。

下面给出整体算法流程的伪代码（中文注释标示对应公式编号）：

算法：HyperGCL 聚类（基于增强超图和隐空间 NCut）

输入：节点属性 X ，超边索引 H ，簇数 K ，超参数 $\alpha, \beta, \gamma, \delta, \lambda_{\text{LC}}, \lambda_{\text{bal}}$ 等

阶段1：预训练

```
for epoch = 1 ... pretrain_epochs:
    构造负超边集  $E^{\wedge-}$                                 # 参见负超边部分
    生成增强视图  $(X^1, H^1), (X^2, H^2)$                 # 特征掩码与超边扰动
     $Z = f_{\theta}(X, H)$                                     # 统一编码器，节点嵌入 公式(18)
     $Z^1 = f_{\theta}(X^1, H^1), Z^2 = f_{\theta}(X^2, H^2)$ 
     $P^1 = g_{\phi}(Z^1), P^2 = g_{\phi}(Z^2)$                     # 投影头输出
     $R = \text{ClusterPrototypes}(Z)$                         # 聚类 logits
    设置  $\gamma=0, \lambda_{\text{LC}}=0$                                 # 关闭聚类与切割损失
    计算  $L_{\text{struct}}$ （公式(2)）， $L_{\text{InfoNCE}}$ （公式(4)）， $L_{\text{intra}}$ （公式(9)）
     $L_{\text{pre}} = \alpha L_{\text{struct}} + \beta L_{\text{InfoNCE}} + \delta L_{\text{intra}}$ 
    反向传播并更新模型参数  $\theta, \phi, C$  等
```

若 `use_kmeans_init`:

```
 $Z_{\text{pre}} = f_{\theta}(X, H)$                                 # 提取预训练嵌入
 $C = \text{KMeans}(Z_{\text{pre}}, K)$                             # 用 KMeans 中心初始化原型
```

阶段2：微调

```
for epoch = 1 ... finetune_epochs:
    构造负超边  $E^{\wedge-}$ 
    生成增强视图  $(X^1, H^1), (X^2, H^2)$ 
     $Z = f_{\theta}(X, H)$ 
     $Z^1 = f_{\theta}(X^1, H^1), Z^2 = f_{\theta}(X^2, H^2)$ 
     $P^1 = g_{\phi}(Z^1), P^2 = g_{\phi}(Z^2)$ 
     $R = \text{ClusterPrototypes}(Z)$ 
     $S = \text{softmax}(R)$                                     # 软聚类分配
    计算  $L_{\text{struct}}, L_{\text{InfoNCE}}, L_{\text{cluster}}$ （参照(6)）， $L_{\text{intra}}$ 
    若  $\lambda_{\text{LC}} > 0$ :
        构造动态图  $A(Z)$ （公式(10)-(11)）                # detach_affinity 控制梯度流
        计算  $L_{\text{cut}}$ （公式(14)）                            # 归一化切割损失
        计算  $L_{\text{balance}}$ （公式(16) 或 (17)）
    优化总损失:  $L_{\text{total}} = \alpha L_{\text{struct}} + \beta L_{\text{InfoNCE}} + \gamma L_{\text{cluster}}$ 
        +  $\delta L_{\text{intra}} + \lambda_{\text{LC}} (L_{\text{cut}} + \lambda_{\text{bal}} L_{\text{balance}})$ 
    更新  $\theta, \phi, C$  等参数
```

输出：最终预测标签 $\hat{y}_{\text{hat}} = \text{argmax}(\text{ClusterPrototypes}(f_{\theta}(X, H)))$ 。

8. 超参数列表

下表列出源码可见的主要超参数及默认值，及其对公式的影响：

参数名称	默认值（示例）	作用 & 敏感性
alpha	1.0	$\mathcal{L}_{\text{struct}}$ 的权重（公式(1)中）
beta	1.0	$\mathcal{L}_{\text{InfoNCE}}$ 权重
gamma	0 或 >0	$\mathcal{L}_{\text{cluster}}$ 权重，影响伪标签学习
delta	1.0	$\mathcal{L}_{\text{intra}}$ 权重
lnahc_weight	0 或 >0	λ_{LC} ，控制是否启用切割损失与平衡损失
ortho_weight	0.1	λ_{bal} ，平衡正交均衡/防空簇损失权重
balance_loss_type	"ortho"/"empty"	平衡损失类型（公式(16) 或 (17)）
use_kmeans_init	True/False	是否使用预训练嵌入 K-Means 初始化聚类原型
detach_affinity	True	是否阻断 $A(Z)$ 梯度（影响梯度路径）
feat_mask_rate	0.3	特征掩码概率，影响数据增强强度
edge_drop_rate	0.5	超边掩码概率，影响数据增强
empty_cluster_rho	0.02	防空簇阈值 ρ （平衡项公式(17)中的值）
归一化温度 τ, τ_c	0.5（示例）	对比损失和聚类 logits 的温度（影响软max曲率）
训练轮数 (pretrain_epochs , finetune_epochs)	—	影响优化阶段区分，决定是否进入微调及损失开启

上述超参数在公式中直接体现为损失的系数或条件判断。尤其 $\gamma, \lambda_{\text{LC}}$ 控制训练阶段损失结构；`detach_affinity` 控制梯度流向；温度 τ 控制软max 的平滑程度；阈值 0.5 和 $\varepsilon = 10^{-8}, 10^{-4}$ 均为代码硬编码，分别用于相似度筛选和矩阵稳定。

9. 代码实现 vs 原论文差异

下表比较了当前实现与 CAHC、LNAHC/LANHC、HOPER 原文在公式层面的主要差异（至少五项）：

模块/项	原论文 (CAHC/LANHC/HOPER)	代码实现
编码器结构	CAHC 原文使用 HGNN+超边自编码器机制（多跳注意力）	使用 EnhancedHGNN 多头注意力，不同于原文中单一架构
结构损失	CAHC 原文结构指导通过超边自编码重构损失，不使用 BCE 判别	使用判别式 BCE 损失（公式(2)）区分真实/伪超边
对比学习	CAHC 也使用对比，HOPER 原文使用多视图对比和特征对比	采用单超图双增强 InfoNCE（公式(4)）；HOPER 的多视图AE未实现
超边内对比（intra）	HOPER 仅提出多视图重建和结构对比，无属性指导；CAHC/LNAHC 无此项	实现了带属性排斥权重的同超边内部对比（公式(9)），以保留异质性
动态图割矩阵	LNAHC 原文使用固定属性-结构随机游走矩阵	根据当前 Z 动态构造余弦图（公式(10)），不是原文的固定矩阵
切割目标	LNAHC 原文优化 HNCut 损失与学习隐表示	直接对 S 做 soft NCut（公式(14)），包括伪逆加稳定项，原文形式不同
聚类原型	CAHC 将聚类和表示联合学习；原文无可学习原型变量	使用可学习原型 C ，并在预训练后进行 K-Means 初始化
平衡损失	LNAHC 及 CAHC 多数只用正交约束；LANHC 偶用空簇惩罚	提供两种选择：正交均衡（公式(16)）或空簇惩罚（公式(17)），代码可配置
伪标签训练	CAHC 等少用伪标签交叉熵；HOPER 专注自监督	增加聚类伪标签损失（公式(6)），针对最大响应 sharpen 聚类分配

以上差异说明：代码实现主要 **借鉴思想** 而非直接复制原论文公式。如同超边对比借鉴 HOPER 思想但在单图上进行，动态图割借鉴 LNAHC 概念但用当前隐空间生成邻接等。具体改动可参见所列条目。

附：公式清单

供论文中直接引用，以下为主要公式的 LaTeX 表示（对应上述编号）：

$$(1) \quad \mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \gamma \mathcal{L}_{\text{cluster}} + \delta \mathcal{L}_{\text{intra}} + \lambda_{\text{LC}} (\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}}). \quad (2) \quad \mathcal{L}_{\text{struct}} = -\frac{1}{2|\mathcal{E}^+|} \sum_{e \in \mathcal{E}^+} \log \sigma(s_e) - \frac{1}{2|\mathcal{E}^-|} \sum_{e \in \mathcal{E}^-} \log (1 - \sigma(s_e)).$$

$$(1) \quad \mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \gamma \mathcal{L}_{\text{cluster}} + \delta \mathcal{L}_{\text{intra}} + \lambda_{\text{LC}} (\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}}).$$

$$(2) \quad \mathcal{L}_{\text{struct}} = -\frac{1}{2|\mathcal{E}^+|} \sum_{e \in \mathcal{E}^+} \log \sigma(s_e) - \frac{1}{2|\mathcal{E}^-|} \sum_{e \in \mathcal{E}^-} \log (1 - \sigma(s_e)).$$

$$(3) \quad M_{ij} = \frac{(\bar{p}_i^{(1)})^\top \bar{p}_j^{(2)}}{\tau}.$$

$$(4) \quad \mathcal{L}_{\text{InfoNCE}} = -\frac{1}{2N} \sum_{i=1}^N \log \frac{\exp(M_{ii})}{\sum_{j=1}^N \exp(M_{ij})} - \frac{1}{2N} \sum_{i=1}^N \log \frac{\exp(M_{ii})}{\sum_{j=1}^N \exp(M_{ji})}.$$

$$(5) \quad \tilde{y}_i = \arg \max_k r_{ik}, \quad (6) \quad \mathcal{L}_{\text{cluster}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(r_{i\tilde{y}_i})}{\sum_{k=1}^K \exp(r_{ik})}.$$

$$(7) \quad a_{ij} = \frac{\bar{z}_i^\top \bar{z}_j}{\tau}, \quad (8) \quad w_{ij} = \frac{1 - b_{ij}}{2}, \quad \omega_{ij} = \begin{cases} 1, & i = j, \\ w_{ij}, & M_{ij}^H = 1, \\ 0, & \text{else.} \end{cases}$$

$$(9) \quad \mathcal{L}_{\text{intra}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(a_{ii})}{\sum_{j=1}^N M_{ij}^H \omega_{ij} \exp(a_{ij}) + \varepsilon}.$$

$$(10) \quad A_{ij} = \begin{cases} (\bar{z}_i^g)^\top \bar{z}_j^g, & i \neq j, (\bar{z}_i^g)^\top \bar{z}_j^g > 0.5, \\ 0, & \text{otherwise,} \end{cases} \quad A_{ii} = 0.$$

$$(11) \quad \tilde{d}_i = d_i + 10^{-4} \frac{1}{N} \sum_{p=1}^N d_p, \quad D = \text{diag}(\tilde{d}_i).$$

$$(12) \quad V = S^\top D S, \quad B = S^\top A S.$$

$$(13) \quad C_{\text{cut}} = S^\top (D - A) S = V - B.$$

$$(14) \quad \mathcal{L}_{\text{cut}} = \text{Tr}[(S^\top D S + 10^{-4} I)^+ S^\top (D - A) S].$$

$$(15) \quad \text{Tr}(S^\top (D - A) S) = \frac{1}{2} \sum_{i,j} A_{ij} \|s_i - s_j\|_2^2.$$

$$(16) \quad \mathcal{L}_{\text{ortho}} = \left\| \frac{1}{N} S^\top S - \frac{1}{K} I_K \right\|_F^2.$$

$$(17) \quad \mathcal{L}_{\text{empty}} = \frac{1}{K} \sum_{k=1}^K [\max(0, \rho - m_k)]^2, \quad m_k = \frac{1}{N} \sum_{i=1}^N S_{ik}.$$

$$(18) \quad \mathcal{L}_{\text{pre}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \delta \mathcal{L}_{\text{intra}}.$$

$$(19) \quad \mathcal{L}_{\text{fine}} = \alpha \mathcal{L}_{\text{struct}} + \beta \mathcal{L}_{\text{InfoNCE}} + \gamma \mathcal{L}_{\text{cluster}} + \delta \mathcal{L}_{\text{intra}} + \lambda_{\text{LC}} (\mathcal{L}_{\text{cut}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}}).$$